

Computer Vision

Lecture 05: 객체 감지용 커스텀 데이터셋 및 모델 학습 파이프라인 구축

Jaesung Lee

curseor@cau.ac.kr

2025. 04. 02.



Contents

1. 데이터셋 다운로드
2. 논문 탐색
3. 데이터셋, 데이터로더 클래스 정의
4. 모델 forward pass 및 loss 계산

1. 데이터셋 다운로드

- 깃허브 리파지토리
 - https://github.com/tkdgur658/CAU_25_Spring_ComputerVision/tree/main/Week05
- 가상환경 필요 라이브러리 설치 (README05.md 참조)

1. 가상환경 생성/활성화

다음 명령어를 통해 Python 3.10을 사용하는 `ComputerVision` 이라는 이름의 가상환경을 생성합니다.

```
conda create -n ComputerVision python=3.10
```



2. 가상환경 활성화

생성한 환경을 활성화합니다.

```
conda activate ComputerVision
```



3. Jupyter Kernel 등록

Jupyter Notebook에서 가상환경을 사용할 수 있도록 커널(Kernel)을 등록합니다.

```
pip install ipykernel  
python -m ipykernel install --user --name ComputerVision --display-name ComputerVision
```



4. 필요 패키지 설치

강의에서 사용하는 필수 패키지를 설치합니다. (`requirements.txt` 파일이 있는 폴더에서 실행하세요.)

```
pip install -r requirements.txt
```



1. 데이터셋 다운로드

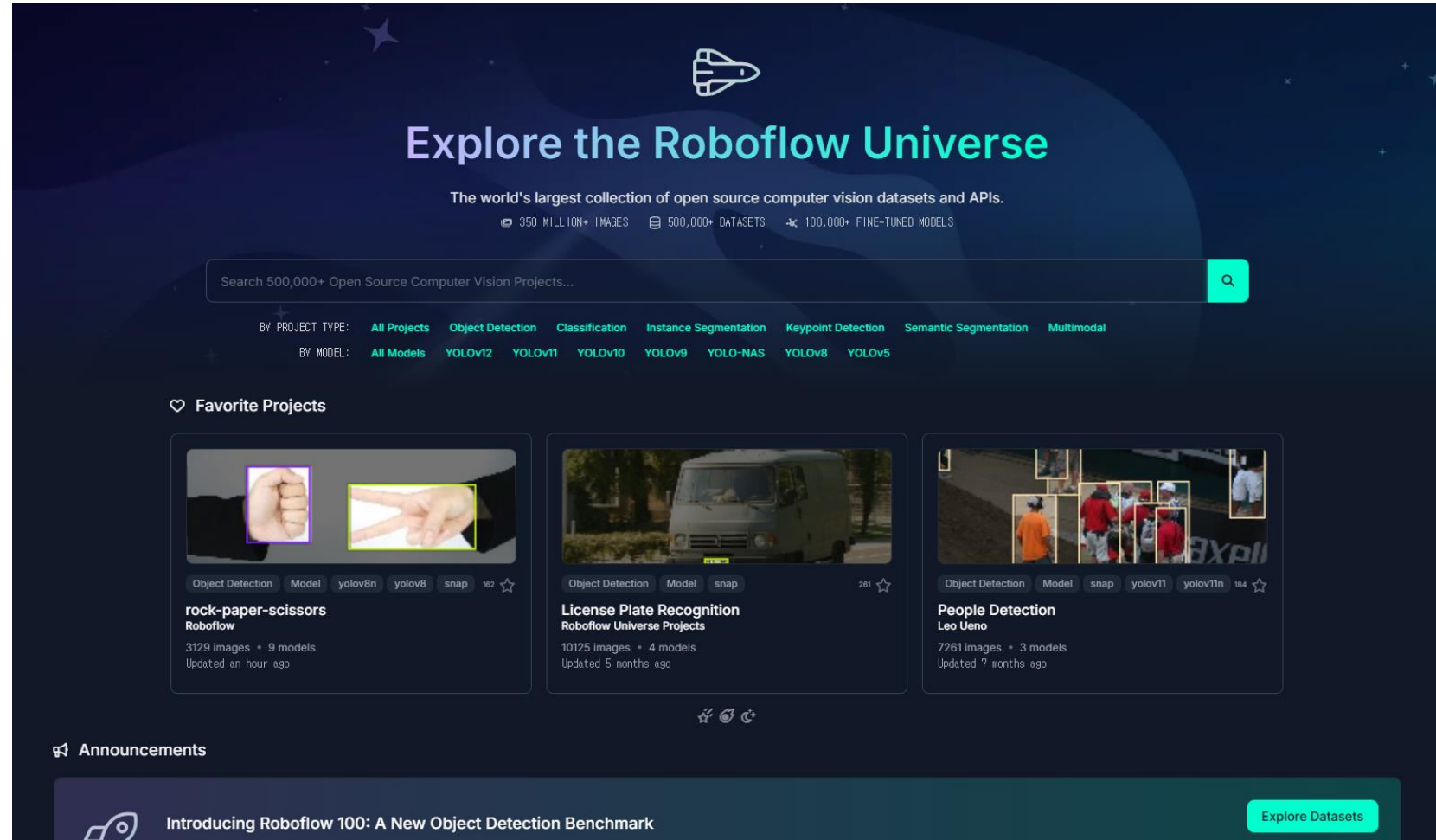
2. 논문 탐색

3. 데이터셋, 데이터로더
클래스 정의

4. 모델 forward pass 및
loss 계산

1. 데이터셋 다운로드

- 실습에 필요한 데이터셋 다운로드를 위한 Roboflow 사이트 접속
 - <https://universe.roboflow.com/>



1. 데이터셋 다운로드

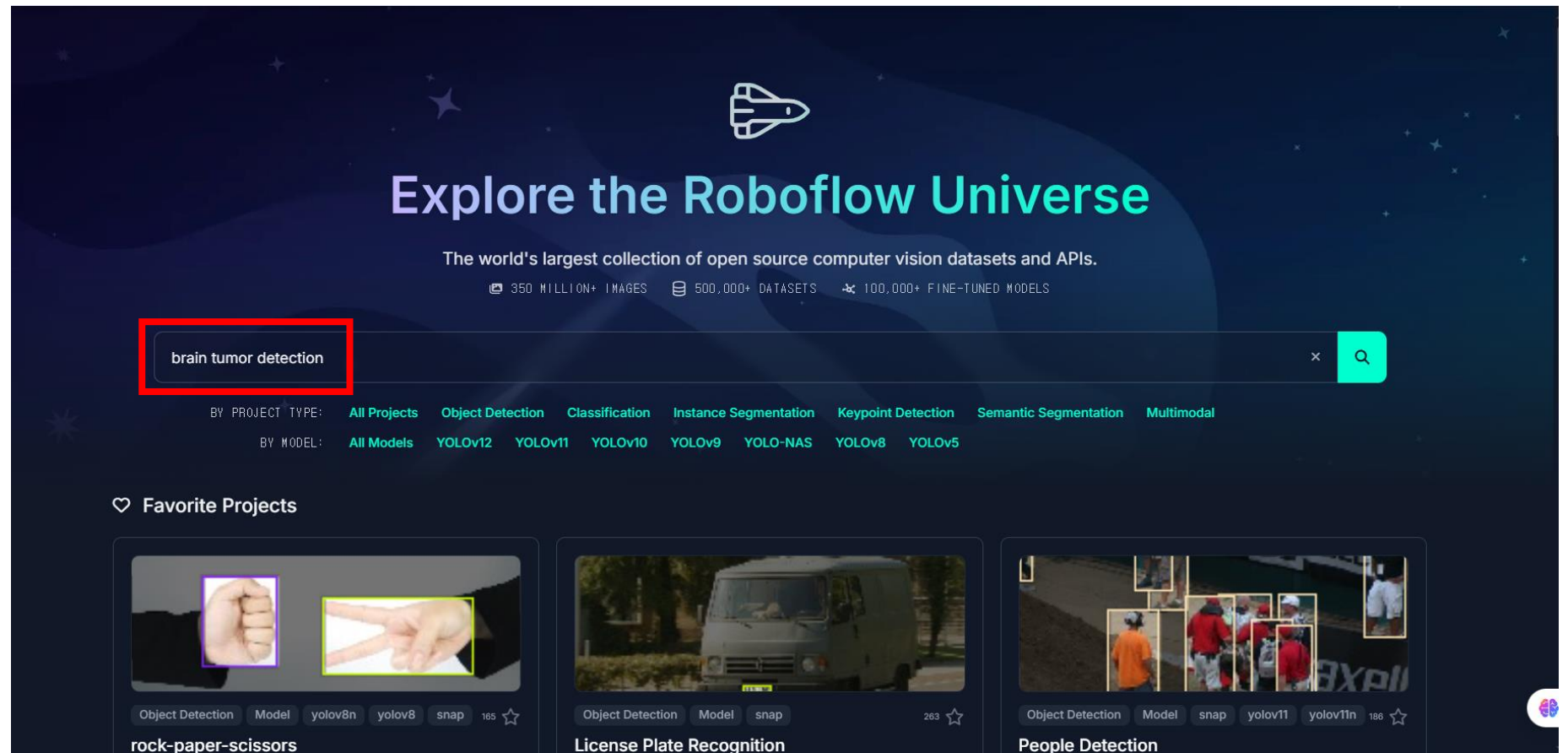
- 검색창에 "Object(원하는 도메인) Detection" 을 입력 후 검색
- 이번 실습에서는 brain tumor detection을 입력

1. 데이터셋 다운로드

2. 논문 탐색

3. 데이터셋, 데이터로더
클래스 정의

4. 모델 forward pass 및
loss 계산



1. 데이터셋 다운로드

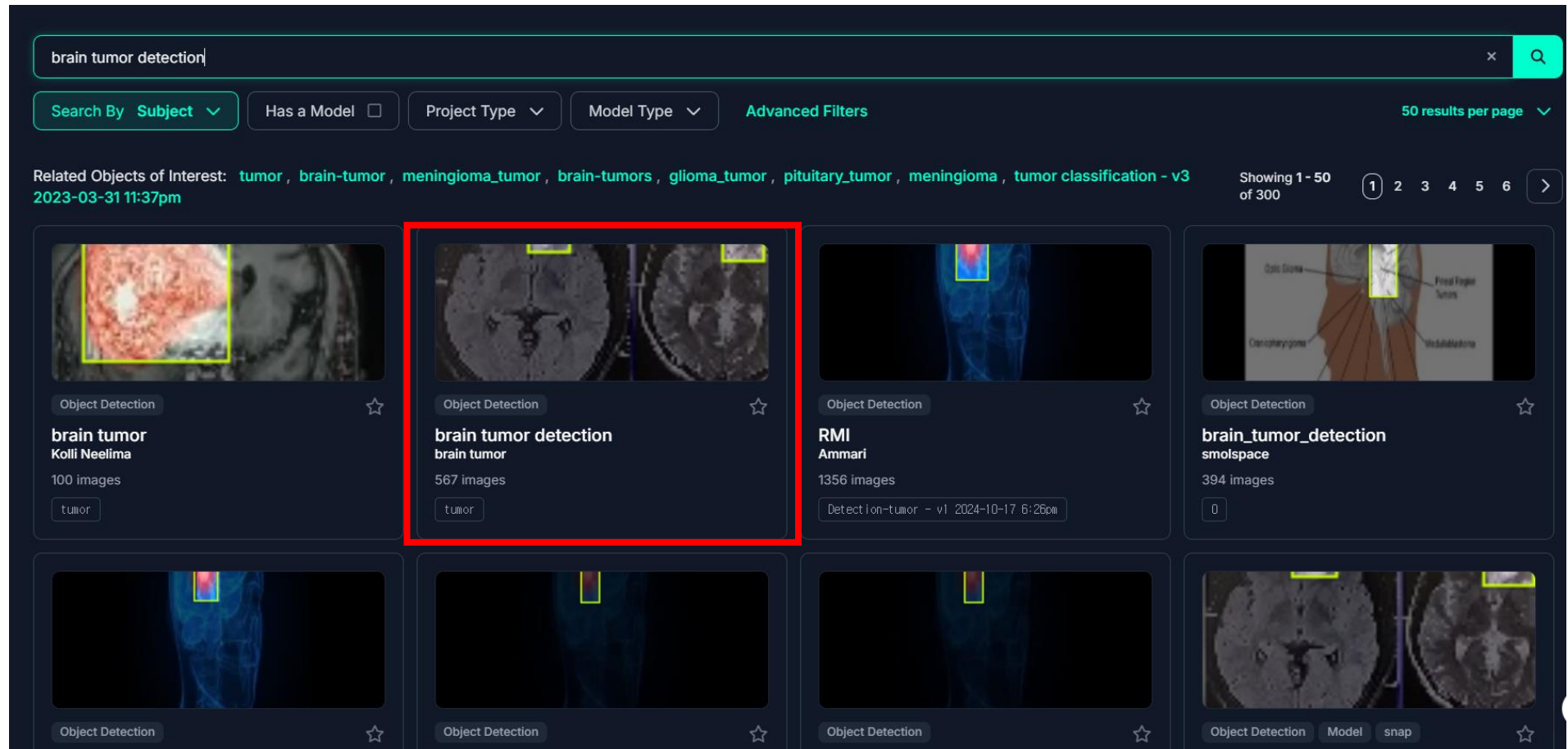
- 여기서는 예시로, “brain tumor detection Computer Vision Project” 을 진행

1. 데이터셋 다운로드

2. 논문 탐색

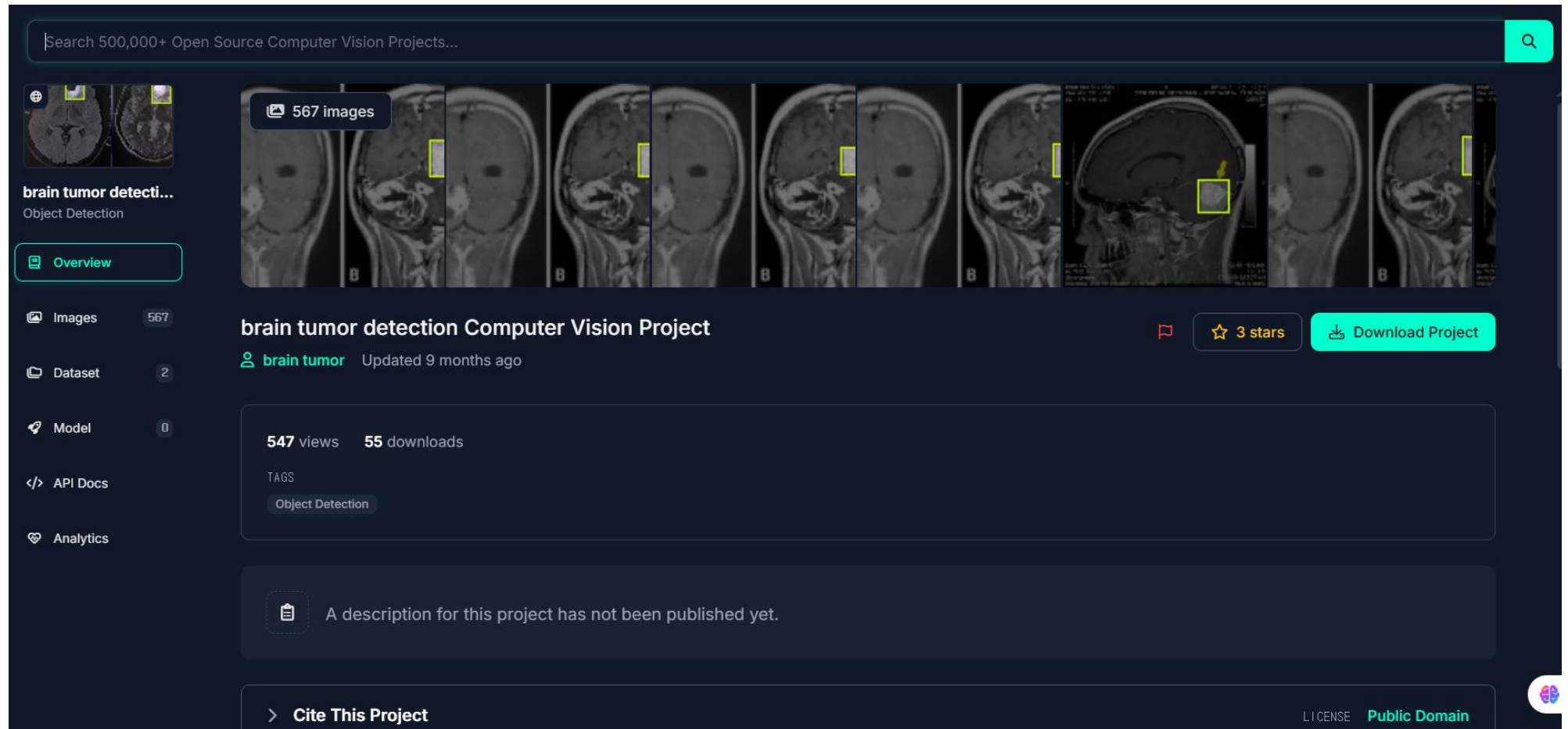
3. 데이터셋, 데이터로더
클래스 정의

4. 모델 forward pass 및
loss 계산



1. 데이터셋 다운로드

- 여기서는 예시로, “brain tumor detection Computer Vision Project” 을 진행



1. 데이터셋 다운로드

- 좌측의 Dataset 을 클릭

1. 데이터셋 다운로드

2. 논문 탐색

3. 데이터셋, 데이터로더
클래스 정의

4. 모델 forward pass 및
loss 계산

The screenshot shows the Hugging Face Dataset Browser interface for the 'brain tumor detection' dataset. The left sidebar contains navigation links: Overview, Images (567), Dataset (highlighted with a red box and a green '2' badge), Model (0), API Docs, and Analytics. The main content area displays the 'Dataset Versions' section, showing two versions: 'mahitha' (v2, 9 months ago) and '2024-07-02 6:33am' (v1, 9 months ago). The 'mahitha' version is highlighted with a green box. Below the versions, there is a 'Popular Download Formats' section with buttons for YOLOv11, YOLOv9, YOLOv8, YOLOv5, YOLOv7, COCO JSON, YOLO Darknet, Pascal VOC XML, TFRecord, PaliGemma, CreateML JSON, and Other Formats. At the bottom, it shows '567 Total Images' and a 'View All Images' link. The dataset is titled 'brain tumor detecti...' and is categorized as 'Object Detection'.

1. 데이터셋 다운로드

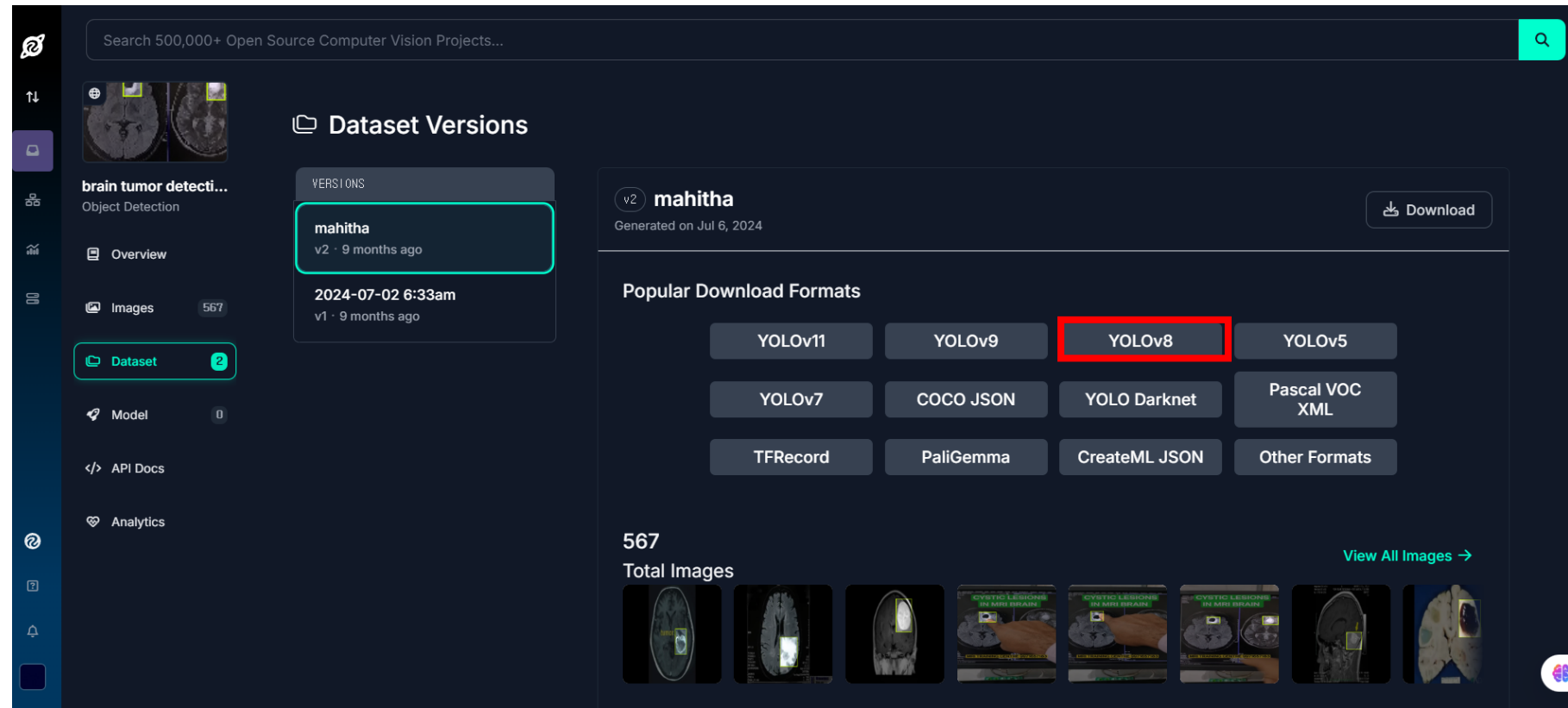
- 이후, 원하는 버전을 선택하고, 데이터 포맷 양식을 선정하여 다운로드

1. 데이터셋 다운로드

2. 논문 탐색

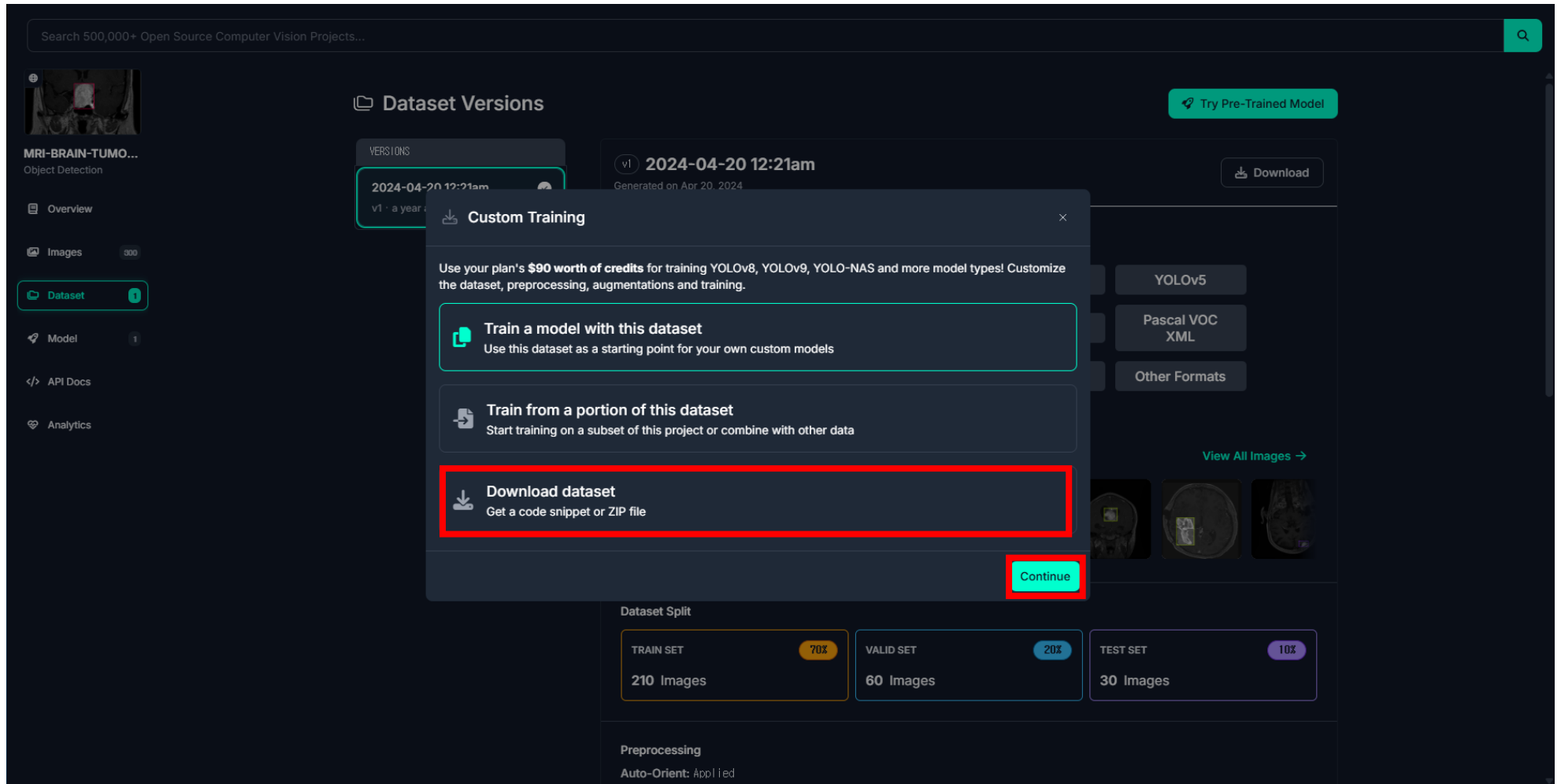
3. 데이터셋, 데이터로더
클래스 정의

4. 모델 forward pass 및
loss 계산



1. 데이터셋 다운로드

- 세 번째의 Download dataset 을 클릭한 후 Continue 버튼을 클릭



1. 데이터셋 다운로드

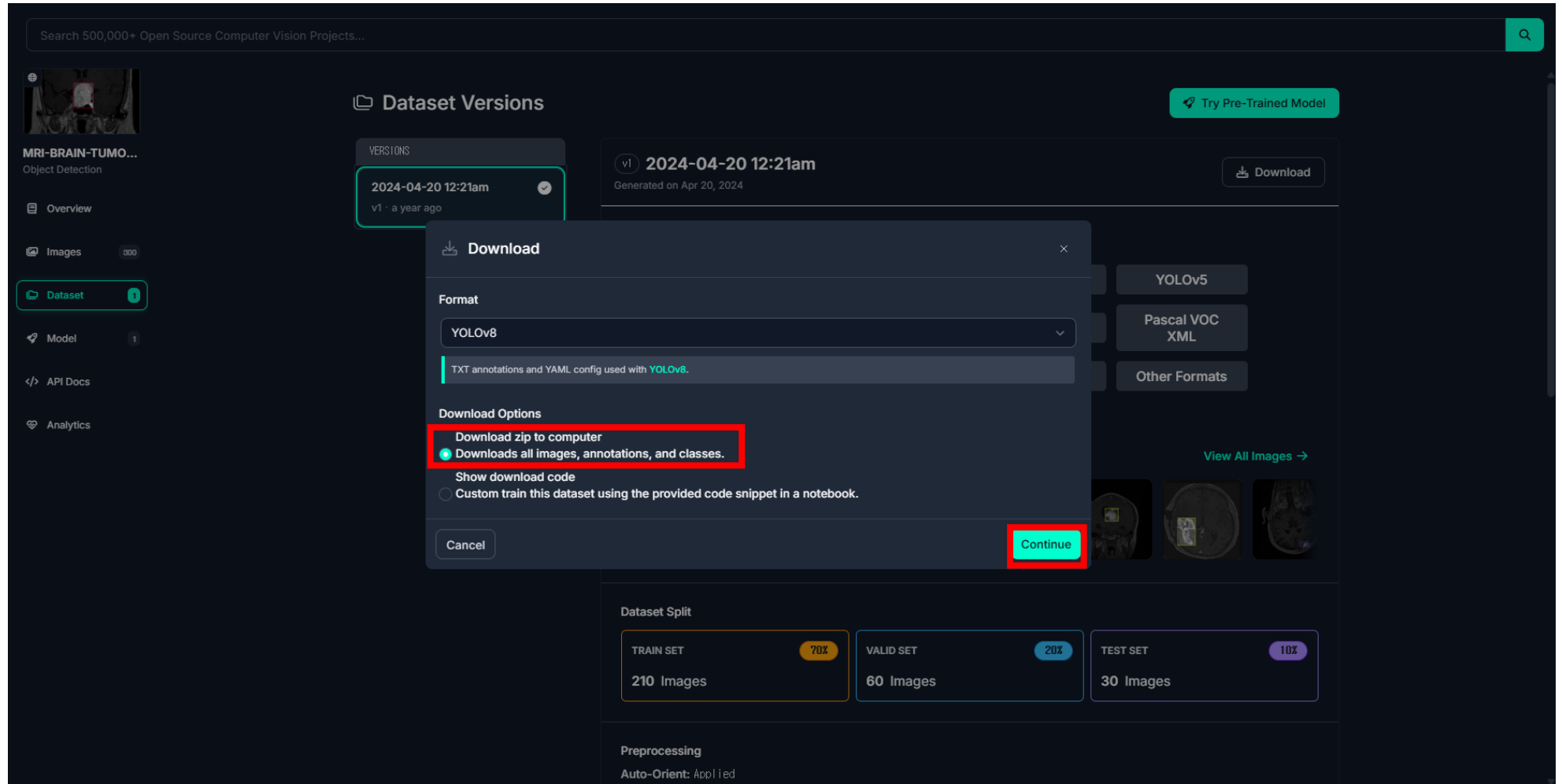
2. 논문 탐색

3. 데이터셋, 데이터로더
클래스 정의

4. 모델 forward pass 및
loss 계산

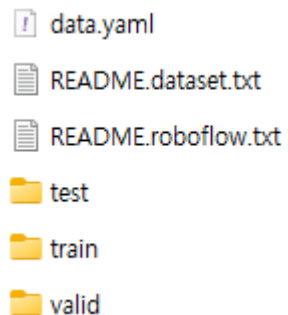
1. 데이터셋 다운로드

- 상단의 Download zip to computer 을 클릭한 후 Continue 버튼을 클릭



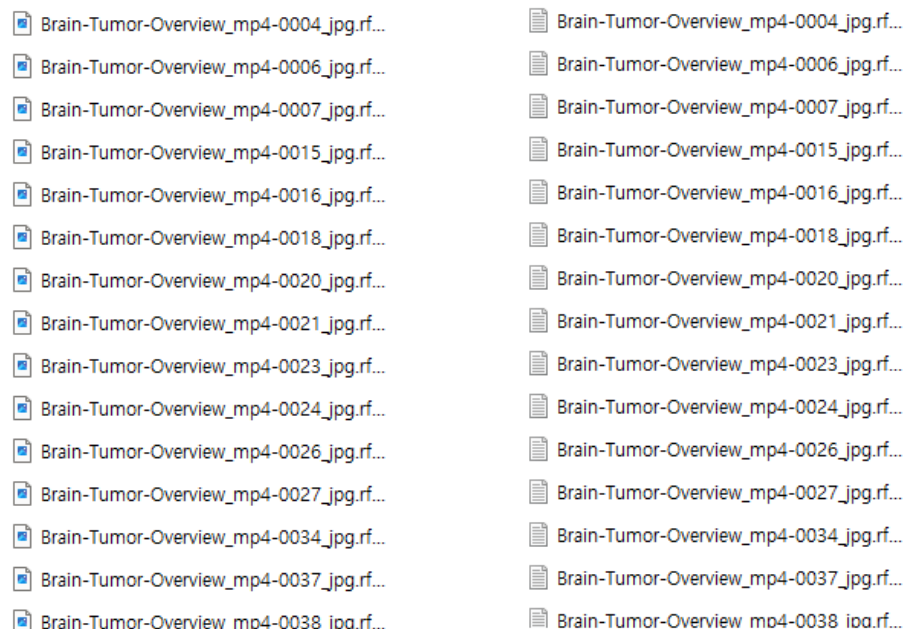
1. 데이터셋 다운로드 및 모델 로드

- 다운로드 된 zip 파일을 압축 해제 한 후, 내부를 살펴보면 train, test, valid 폴더로 나누어져 있음



data.yaml
README.dataset.txt
README.roboflow.txt
test
train
valid

- 각 폴더 안에는 images와 labels 폴더 안에 각각 이미지 데이터와 텍스트 파일로 구성되어 있음



Brain-Tumor-Overview_mp4-0004.jpg.rf...
Brain-Tumor-Overview_mp4-0006.jpg.rf...
Brain-Tumor-Overview_mp4-0007.jpg.rf...
Brain-Tumor-Overview_mp4-0015.jpg.rf...
Brain-Tumor-Overview_mp4-0016.jpg.rf...
Brain-Tumor-Overview_mp4-0018.jpg.rf...
Brain-Tumor-Overview_mp4-0020.jpg.rf...
Brain-Tumor-Overview_mp4-0021.jpg.rf...
Brain-Tumor-Overview_mp4-0023.jpg.rf...
Brain-Tumor-Overview_mp4-0024.jpg.rf...
Brain-Tumor-Overview_mp4-0026.jpg.rf...
Brain-Tumor-Overview_mp4-0027.jpg.rf...
Brain-Tumor-Overview_mp4-0034.jpg.rf...
Brain-Tumor-Overview_mp4-0037.jpg.rf...
Brain-Tumor-Overview mp4-0038 iqa.rf...

2. 논문 탐색

- Google Scholar 접속 (<https://scholar.google.com/>)

☰ 내 프로필 ★ 내 서재



Google 학술검색



☒ 모든 언어 ☐ 한국어 웹

거인의 어깨에 올라서서 더 넓은 세상을 바라보라 - 아이작 뉴턴

🌐 KO

[개인정보처리방침](#) [약관](#) [도움말](#)

2. 논문 탐색

- 기간 설정 (2021년도 부터)

≡ Google 학술검색

brain tumor detection github

Q

📌 학술자료

검색결과 약 17,500개 중 2페이지 (0.11초)

모든 날짜

2025 년부터

2024 년부터

2021 년부터

기간 설정...

관련도별 정렬

날짜별 정렬

모든 언어

한국어 웹

모든 유형

검토 자료

☐ 특허 포함

☒ 서지정보 포함

☒ 알림 만들기

Bitr-unet: a cnn-transformer combined network for mri brain tumor segmentation [PDF] nih.gov

[Q Jia, H Shu](#) - International MICCAI brainlesion workshop, 2021 - Springer

... Therefore, we propose a CNN-Transformer combined model, called BiTr-Unet, with specific modifications for brain tumor segmentation on multi-modal MRI scans. Our BiTr-Unet ...

☆ 저장 📄 인용 123회 인용 관련 학술자료 전체 9개의 버전

RCS-YOLO: A fast and high-accuracy object detector for brain tumor detection [PDF] arxiv.org

[M Kang, CM Ting, FF Ting, RCW Phan](#) - International conference on ..., 2023 - Springer

... The code is available at <https://github.com/mkang315/...> brain tumor detection. To our best knowledge, this is the first work to leverage on YOLO-based model for fast brain tumor detection. ...

☆ 저장 📄 인용 58회 인용 관련 학술자료 전체 8개의 버전

Bgf-yolo: Enhanced yolov8 with multiscale attentional feature fusion for brain tumor detection [PDF] arxiv.org

[M Kang, CM Ting, FF Ting, RCW Phan](#) - International Conference on ..., 2024 - Springer

... feature fusions, detection head architectures on brain tumor detection accuracy. Experimental ... -art on the brain tumor detection dataset Br35H. The code is available at <https://github.com/...>

☆ 저장 📄 인용 33회 인용 관련 학술자료 전체 5개의 버전

Image segmentation using Mask R-CNN for tumor detection from medical images [PDF] ieee.org

[T Padma, CU Kumari, D Yamini](#)... - ... on Electronics and ..., 2022 - ieeexplore.ieee.org

... The Mask R-CNN repository is downloaded from GitHub. The repository contains some ...

Image analysis for MRI based brain tumor detection and feature extraction using biologically ...

☆ 저장 📄 인용 17회 인용 관련 학술자료

[HTML] Integrating Competitive Framework into Differential Evolution: Comprehensive performance analysis and application in brain tumor detection [HTML] sciencedirect.com

CAU AutoML Lab

14

2. 논문 탐색

- 이 실습에서는 RCS-YOLO 논문을 선정하여 진행

RCS-YOLO: A fast and high-accuracy object detector for brain tumor detection

[PDF] [arxiv.org](#)

[M Kang](#), [CM Ting](#), [FF Ting](#), [RCW Phan](#) - International conference on ..., 2023 - Springer

... We apply the proposed RCS-YOLO model for a challenging task of brain tumor detection.

To our best knowledge, this is the first work to leverage on YOLO-based model for fast brain ...

☆ 저장 99 인용 58회 인용 관련 학술자료 전체 8개의 버전



RCS-YOLO: A Fast and High-Accuracy Object Detector for Brain Tumor Detection

Ming Kang[✉], Chee-Ming Ting[✉], Fung Fung Ting[✉],
and Raphaël C.-W. Phan[✉]

School of Information Technology, Monash University, Malaysia Campus,
Subang Jaya, Malaysia
ting.cheeming@monash.edu

Abstract. With an excellent balance between speed and accuracy, cutting-edge YOLO frameworks have become one of the most efficient algorithms for object detection. However, the performance of using YOLO networks is scarcely investigated in brain tumor detection. We propose a novel YOLO architecture with Reparameterized Convolution based on channel Shuffle (RCS-YOLO). We present RCS and a One-Shot Aggregation of RCS (RCS-OSA), which link feature cascade and computation efficiency to extract richer information and reduce time consumption. Experimental results on the brain tumor dataset Br35H show that the proposed model surpasses YOLOv6, YOLOv7, and YOLOv8 in speed and accuracy. Notably, compared with YOLOv7, the precision of RCS-YOLO improves by 1%, and the inference speed by 60% at 114.8 images detected per second (FPS). Our proposed RCS-YOLO achieves state-of-the-art performance on the brain tumor detection task. The code is available at <https://github.com/mkang315/RCS-YOLO>.

Keywords: Medical image detection · YOLO · Reparameterized convolution · Channel shuffle · Computation efficiency

1 Introduction

3. 데이터셋, 데이터로더 클래스 정의

- 데이터셋과 모델 파일을 다음과 같이 프로젝트 경로에 정리

/ ... / CAU_25_Spring_ComputerVision-main / Week05 /

☐ Name	Last Modified	File Size
brain tumor detection.v2-mahitha.yolov8	45 minutes ago	
cfg	2 months ago	
data	17 minutes ago	
models	17 minutes ago	
utils	16 minutes ago	
01_Custom_Dataset_and_Model.ipynb	1 minute ago	908.3 KB
brain tumor detection.v2-mahitha.yolov8.zip	45 minutes ago	24.9 MB
data_utils.py	21 minutes ago	10.2 KB
RCS-YOLO-main.zip	39 minutes ago	18.4 MB
README05.md	1 hour ago	389 B
requirements.txt	1 hour ago	491 B

/ ... / Week05 / brain tumor detection.v2-mahitha.yolov8 /

☐ Name	Last Modified	File Size
test	8 months ago	
train	8 months ago	
valid	8 months ago	
data.yaml	8 months ago	300 B
README.dataset.txt	8 months ago	163 B
README.roboflow.txt	8 months ago	983 B



1. Check Custom Dataset and Model

```
[1]: 1 import os
2 base_data_dir = 'brain tumor detection.v2-mahitha.yolov8'
3 base_model_dirs = ['models', 'cfg', 'data', 'utils']
4 print(os.path.exists(base_data_dir))
5 for dir in base_model_dirs:
6     print(os.path.exists(dir))

Last executed at 2025-04-02 08:27:30 in 14ms

True
True
True
True
True
```


3. 데이터셋, 데이터로더 클래스 정의

- train/images, train/labels, valid/image, valid/labels 경로 확인
- 커스텀 Dataset 클래스 정의

2. Dataset Class, DataLoader Definition

```
[2]: 1 from data_utils import T, YOLOv8Dataset, DataLoader, yolo_collate_fn, visualize_batch
2
3 train_img_dir = os.path.join(base_data_dir, "train", "images")
4 train_label_dir = os.path.join(base_data_dir, "train", "labels")
5 val_img_dir = os.path.join(base_data_dir, "valid", "images")
6 val_label_dir = os.path.join(base_data_dir, "valid", "labels")
7 # Add test paths similarly if needed
8 # test_img_dir = os.path.join(base_data_dir, "test", "images")
9 # test_label_dir = os.path.join(base_data_dir, "test", "labels")
10 # -----
11 # Check if directories exist
12 if os.path.isdir(train_img_dir):
13     print(f"Train image directory: {train_img_dir}")
14 else:
15     print(f"Error: Train image directory not found: {train_img_dir}")
16 if os.path.isdir(train_label_dir):
17     print(f"Train label directory: {train_label_dir}")
18 else:
19     print(f"Error: Train label directory not found: {train_label_dir}")
20 if os.path.isdir(val_img_dir):
21     print(f"Validation image directory: {val_img_dir}")
22 else:
23     print(f"Warning: Validation image directory not found: {val_img_dir}")
24 if os.path.isdir(val_label_dir):
25     print(f"Validation label directory: {val_label_dir}")
26 else:
27     print(f"Warning: Validation label directory not found: {val_label_dir}")
```

Last executed at 2025-04-02 08:27:32 in 12ms

Train image directory: brain tumor detection.v2-mahitha.yolov8\train\images
Train label directory: brain tumor detection.v2-mahitha.yolov8\train\labels
Validation image directory: brain tumor detection.v2-mahitha.yolov8\valid\images
Validation label directory: brain tumor detection.v2-mahitha.yolov8\valid\labels

```
[3]: 1 # Define image transformations
2 transform = T.Compose([
3     T.Resize((640, 640)), # Typical input size for YOLOv8
4     T.ToTensor(),
5 ])
6
7 # Create Train Dataset and DataLoader
8 train_dataset = YOLOv8Dataset(img_dir=train_img_dir,
9                               label_dir=train_label_dir,
10                              transform=transform)
11 print(len(train_dataset))
```

Last executed at 2025-04-02 08:43:20 in 29ms

Debug: Found 397 potential image files in brain tumor detection.v2-mahitha.yolov8\train\images
Debug: Found 397 potential label files in brain tumor detection.v2-mahitha.yolov8\train\labels
Debug: Matched 397 image-label pairs.
397

3. 데이터셋, 데이터로더 클래스 정의

- Training용 커스텀 DataLoader 정의 및 첫번째 미니배치 정보 확인

```
[4]: 1 train_loader = DataLoader(train_dataset,
2                               batch_size=4,      # Reduced batch size for testing
3                               shuffle=True,      # Shuffle training data
4                               num_workers=0,     # Set to 0 for debugging, especially on Windows
5                               collate_fn=yolo_collate_fn, # Use custom collate function
6                               pin_memory=True)   # Optimize data transfer to GPU if available
7
8 print(f"Train dataset loaded successfully. Total samples: {len(train_dataset)}")
9
10 # Test loading a batch from the DataLoader
11 for i, batch_data in enumerate(train_loader):
12     if batch_data is None or batch_data[0] is None:
13         print(f"--- Failed to load Batch {i+1} (collate_fn returned None) ---")
14         continue # Try next batch
15
16     images, labels = batch_data
17     print(f"\n--- Batch {i+1} ---")
18     print(f"Images batch shape: {images.shape}") # Expected: [batch_size, C, H, W]
19     print(f"Labels batch length: {len(labels)}") # Expected: batch_size (List of tensors)
20     for j, label_tensor in enumerate(labels):
21         print(f"  Label tensor {j} shape: {label_tensor.shape}") # Expected: [Num_objects_in_img_j, 5]
22
23         # Print the actual labels (up to 5 for readability)
24         max_labels_to_show = min(5, label_tensor.shape[0])
25         for k in range(max_labels_to_show):
26             cls_id, x, y, w, h = label_tensor[k].tolist()
27             print(f"    Box {k}: class={int(cls_id)}, x={x:.4f}, y={y:.4f}, w={w:.4f}, h={h:.4f}")
28
29         # If there are more labels than shown, indicate this
30         if label_tensor.shape[0] > max_labels_to_show:
31             print(f"    ... and {label_tensor.shape[0] - max_labels_to_show} more boxes")
32     visualize_batch(images, labels)
33     if i == 0:
34         break # Only check the first batch
```

Last executed at 2025-04-02 09:32:56 in 464ms

Train dataset loaded successfully. Total samples: 397

1. 데이터셋 다운로드

2. 논문 탐색

3. 데이터셋, 데이터로더
클래스 정의

4. 모델 forward pass 및
loss 계산

3. 데이터셋, 데이터로더 클래스 정의

- Training용 커스텀 DataLoader 정의 및 첫번째 미니배치 정보 확인

```
--- Batch 1 ---  
Images batch shape: torch.Size([4, 3, 640, 640])  
Labels batch length: 4  
Label tensor 0 shape: torch.Size([1, 5])  
Box 0: class=0, x=0.6313, y=0.5359, w=0.0938, h=0.2672  
Label tensor 1 shape: torch.Size([1, 5])  
Box 0: class=0, x=0.6359, y=0.5688, w=0.0883, h=0.2625  
Label tensor 2 shape: torch.Size([1, 5])  
Box 0: class=0, x=0.5969, y=0.5203, w=0.0930, h=0.2977  
Label tensor 3 shape: torch.Size([2, 5])  
Box 0: class=0, x=0.2992, y=0.3227, w=0.1984, h=0.1227  
Box 1: class=0, x=0.8852, y=0.3391, w=0.1289, h=0.0883
```

Image 1

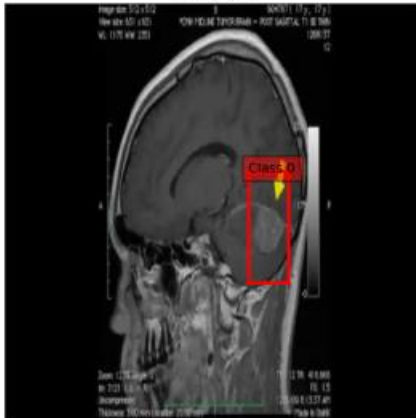


Image 2



Image 3

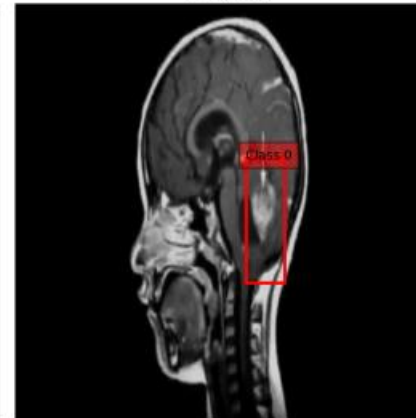


Image 4



3. 데이터셋, 데이터로더 클래스 정의

- Validation용 커스텀 DataLoader 정의

```
[5]: 1 # Create Validation Dataset and DataLoader
2 # Only create validation loader if directories exist
3 if os.path.isdir(val_img_dir) and os.path.isdir(val_label_dir):
4     val_dataset = YOLOv8Dataset(img_dir=val_img_dir,
5                                 label_dir=val_label_dir,
6                                 transform=transform)
7
8     if len(val_dataset) > 0:
9         val_loader = DataLoader(val_dataset,
10                                 batch_size=4,
11                                 shuffle=False, # No need to shuffle validation data
12                                 num_workers=0,
13                                 collate_fn=yolo_collate_fn,
14                                 pin_memory=True)
15         print(f"\nValidation dataset loaded successfully. Total samples: {len(val_dataset)}")
16     else:
17         print("Warning: No valid samples found after creating Validation dataset.")
18 else:
19     print("Warning: Validation directories not valid. Skipping Validation DataLoader creation.")
```

Last executed at 2025-04-02 09:32:56 in 13ms

Debug: Found 114 potential image files in brain tumor detection.v2-mahitha.yolov8\valid\images
Debug: Found 114 potential label files in brain tumor detection.v2-mahitha.yolov8\valid\labels
Debug: Matched 114 image-label pairs.

Validation dataset loaded successfully. Total samples: 114

3. 데이터셋, 데이터로더 클래스 정의

- Validation용 커스텀 DataLoader 정의

3. Load Model & Forward pass

```
[6]: 1 import torch
2 import yaml
3
4 # --- 1. Manually Set Command-Line Arguments ---
5 class DummyOpt:
6     pass
7
8 opt = DummyOpt()
9 opt.workers = 8
10 opt.device = "cpu" # Set to "cpu" if GPU is not available
11 opt.batch_size = 32
12 opt.data = "data/br35h.yaml" # Data configuration file (includes number of classes, names, etc.)
13 opt.img_size = [640, 640] # Image sizes for [train, test]
14 opt.cfg = "cfg/training/rcs-yolo.yaml" # Model configuration file
15 opt.weights = "" # If empty, create model from scratch (no pretrained weights)
16 opt.name = "rcs-yolo"
17 opt.hyp = "data/hyp_training.yaml" # Hyperparameter file
18 opt.single_cls = False # Whether to treat data as a single class
19
20 # --- 2. Load Device, Hyperparameters, and Dataset Information ---
21 device = torch.device("cuda:" + opt.device if torch.cuda.is_available() else "cpu")
22 print("Device:", device)
23
24 with open(opt.hyp, 'r') as f:
25     hyp = yaml.safe_load(f)
26 print("Hyperparameters loaded from", opt.hyp)
27
28 with open(opt.data, 'r') as f:
29     data_dict = yaml.safe_load(f)
30 nc = 1 if opt.single_cls else int(data_dict['nc'])
31 names = data_dict['names']
32 print(f"Number of classes: {nc}, names: {names}")
```

Last executed at 2025-04-02 09:32:56 in 44ms

Device: cpu
Hyperparameters loaded from data/hyp_training.yaml
Number of classes: 1, names: ['Brain Tumor']

1. 데이터셋 다운로드

2. 논문 탐색

3. 데이터셋, 데이터로더
클래스 정의

4. 모델 forward pass 및
loss 계산

4. 모델 forward pass 및 loss 계산

- 가져온 모델 모듈 선언에 필요한 Option 클래스 생성
 - Device, Batch size, Image size 등 초매개변수 포함

3. Load Model & Forward pass

```
[6]: 1 import torch
2 import yaml
3
4 # --- 1. Manually Set Command-Line Arguments ---
5 class DummyOpt:
6     pass
7
8 opt = DummyOpt()
9 opt.workers = 8
10 opt.device = "cpu" # Set to "cpu" if GPU is not available
11 opt.batch_size = 4
12 opt.data = "data/br35h.yaml" # Data configuration file (includes number of classes, names, etc.)
13 opt.img_size = [640, 640] # Image sizes for [train, test]
14 opt.cfg = "cfg/training/rcs-yolo.yaml" # Model configuration file
15 opt.weights = "" # If empty, create model from scratch (no pretrained weights)
16 opt.name = "rcs-yolo"
17 opt.hyp = "data/hyp_training.yaml" # Hyperparameter file
18 opt.single_cls = False # Whether to treat data as a single class
19
20 # --- 2. Load Device, Hyperparameters, and Dataset Information ---
21 device = torch.device("cuda:" + opt.device if torch.cuda.is_available() else "cpu")
22 print("Device:", device)
23
24 with open(opt.hyp, 'r') as f:
25     hyp = yaml.safe_load(f)
26 print("Hyperparameters loaded from", opt.hyp)
27
28 with open(opt.data, 'r') as f:
29     data_dict = yaml.safe_load(f)
30 nc = 1 if opt.single_cls else int(data_dict['nc'])
31 names = data_dict['names']
32 print(f"Number of classes: {nc}, names: {names}")
```

Last executed at 2025-04-02 09:32:56 in 44ms

Device: cpu

Hyperparameters loaded from data/hyp_training.yaml

Number of classes: 1, names: ['Brain Tumor']

4. 모델 forward pass 및 loss 계산

- Option 객체를 통해 가져온 모델 모듈 선언
- 선언된 Training용 Loader의 1번 배치 확인
- 레이블 배열에 이미지 index 포함 후 이미지 단위에서 객체 단위로 분할

```
[7]: 1 # --- 3. Load Model (Using Model class from models/yolo.py) ---
2 from models.yolo import Model
3 model = Model(opt.cfg, ch=3, nc=nc).to(device)
4 model.train() # Set model to training mode
5 print("Model loaded from", opt.cfg)
6
7 # Assign additional attributes required by ComputeLoss (e.g., hyp and gr)
8 model.hyp = hyp
9 model.gr = 1.0
10
11 # --- 4. Get a Batch from the DataLoader ---
12 # Assume that a custom DataLoader (train_loader) has already been created.
13 batch = next(iter(train_loader))
14 images, labels = batch
15 print("\nUsing batch from train_loader:")
16 print(f"Images batch shape: {images.shape}")
17 print(f"Number of label tensors: {len(labels)}")
18 for j, label_tensor in enumerate(labels):
19     print(f"Label tensor {j} shape: {label_tensor.shape}")
```

Last executed at 2025-04-02 09:32:58 in 1.75s

Model loaded from cfg/training/rcs-yolo.yaml

```
Using batch from train_loader:
Images batch shape: torch.Size([4, 3, 640, 640])
Number of label tensors: 4
Label tensor 0 shape: torch.Size([1, 5])
Label tensor 1 shape: torch.Size([2, 5])
Label tensor 2 shape: torch.Size([0, 5])
Label tensor 3 shape: torch.Size([1, 5])
```

```
[8]: 1 # --- 5. Construct Targets Tensor ---
2 # For YOLO loss, each label needs to include the image index.
3 all_targets = []
4 for i, lab in enumerate(labels):
5     if lab.shape[0] > 0:
6         image_idx = torch.full((lab.shape[0], 1), i, dtype=lab.dtype, device=lab.device)
7         all_targets.append(torch.cat((image_idx, lab), dim=1))
8 if all_targets:
9     targets_tensor = torch.cat(all_targets, dim=0) # Shape: [total_objects, 6]
10 else:
11     targets_tensor = torch.empty((0, 6), device=device)
12 print(f"Constructed targets tensor of shape: {targets_tensor.shape}")
13 print(targets_tensor)
```

Last executed at 2025-04-02 09:32:58 in 13ms

```
Constructed targets tensor of shape: torch.Size([4, 6])
tensor([[0.0000, 0.0000, 0.9125, 0.3343, 0.1742, 0.0835],
        [1.0000, 0.0000, 0.1398, 0.3710, 0.0851, 0.1703],
        [1.0000, 0.0000, 0.8593, 0.4953, 0.1351, 0.2015],
        [3.0000, 0.0000, 0.6445, 0.5703, 0.1093, 0.1992]])
```


4. 모델 forward pass 및 loss 계산

- Forward 실행(이미지를 모델에 입력)
- 손실 함수 계산
 - Box Loss
 - Object Loss
 - Classification Loss
 - Total Loss
- Backward 실행(기울기 계산)
 - backward() 이후 파라미터에 grad가 생성됨

```
[15]: 1 # --- 6. Run Model Forward Pass ---
2 outputs = model(images)
3 print("--- Forward Pass ---")
4 if isinstance(outputs, (list, tuple)):
5     for idx, output in enumerate(outputs):
6         print(f"Output {idx} shape: {output.shape}, sum: {output.sum().item()}")
7 else:
8     print("Output shape:", outputs.shape, "sum:", outputs.sum().item())
9
10 # --- 7. Compute Loss (Using ComputeLoss) ---
11 from utils.loss import ComputeLoss
12 compute_loss = ComputeLoss(model) # ComputeLoss is defined in utils/loss.py
13 loss, loss_items = compute_loss(outputs, targets_tensor)
14 # Loss items: [box_loss, obj_loss, cls_loss, total_loss]
15 print("\nLoss components:")
16 print(f"Box Loss: {loss_items[0].item()}")
17 print(f"Obj Loss: {loss_items[1].item()}")
18 print(f"Cls Loss: {loss_items[2].item()}")
19 print(f"Total Loss: {loss_items[3].item()}")
20
21 print("Before backward() call. Gradient values:")
22 for name, param in model.named_parameters():
23     if param.grad is not None:
24         grad_norm = param.grad.norm().item()
25         print(f"{name} gradient norm: {grad_norm}")
26     break
27 loss.backward()
28 print("\nAfter backward() call. Gradient values:")
29 for name, param in model.named_parameters():
30     if param.grad is not None:
31         grad_norm = param.grad.norm().item()
32         print(f"{name} gradient norm: {grad_norm}")
33     break
34
35 print("\nForward and backward pass with ComputeLoss completed successfully.")
```

Last executed at 2025-04-02 09:47:07 in 3.95s

--- Forward Pass ---
Output 0 shape: torch.Size([4, 2, 40, 40, 6]), sum: -17708.54296875
Output 1 shape: torch.Size([4, 2, 20, 20, 6]), sum: 1611.892822265625

Loss components:
Box Loss: 0.07499455660581589
Obj Loss: 0.10834257304668427
Cls Loss: 0.0
Total Loss: 0.18333712220191956

Before backward() call. Gradient values:

After backward() call. Gradient values:
model.0.rbr_dense.conv.weight gradient norm: 2.1422314643859863

Forward and backward pass with ComputeLoss completed successfully.

Thank you